SQL FINAL REPORT

Employee Database – SQL Queries and Results

Student Name	ATTSHAYA J
Register Number	311423243006
Course / Subject	Structured Query Language
Institution	Meenakshi College Of Engineering
Academic Year	2026-2027

1. Introduction

This report documents the SQL practical work shown in the supplied screenshots. The work uses an employee table and demonstrates table creation, data insertion, data retrieval, filtering, sorting, aggregation, subqueries, and updating records. Each screenshot is included with a short description explaining the purpose of the SQL statement and the result displayed in the SQL client.

2. Objectives

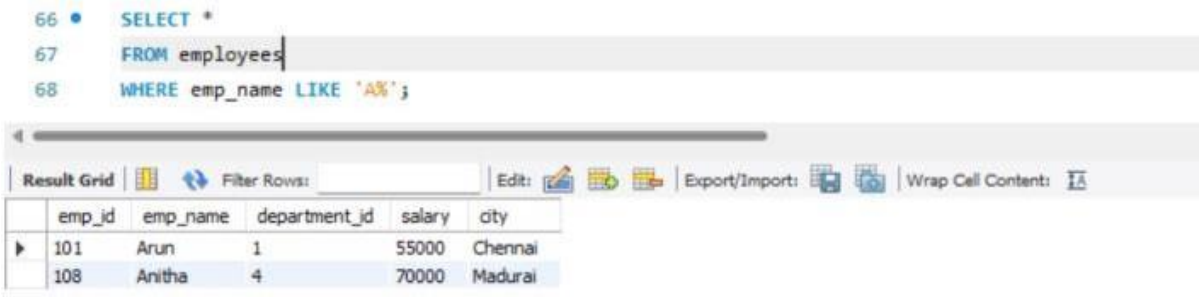
- Create an employee table with relevant employee attributes.
- Insert employee records into the table.
- Retrieve selected columns and complete employee records.
- Filter records using LIKE, BETWEEN, and comparison conditions.
- Calculate aggregate information such as total salary.
- Use a subquery to identify the employee with the maximum salary.
- Sort employee records by salary.
- Update an employee's salary using an UPDATE statement.

3. Database / Table Used

The screenshots show an employee table containing fields such as ID, Name, Department, Salary, Age, City, and Year. The data includes employees from departments including IT, HR, Sales, and Finance, with employee records displayed in the result grids.

4. SQL Practical Work and Screenshot Descriptions

Screenshot 1 – LIKE operator



```
66 •  SELECT *
67    FROM employees
68    WHERE emp_name LIKE 'A%';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

emp_id	emp_name	department_id	salary	city
101	Arun	1	55000	Chennai
108	Anitha	4	70000	Madurai

Description: The query uses SELECT * FROM employees with a WHERE condition using the LIKE operator: emp_name LIKE 'A%'. The percent sign (%) represents any sequence of characters after the letter A. The result grid displays the employees whose names begin with A, including Arun and Anitha. This demonstrates pattern-based filtering of text values.

Screenshot 2 – BETWEEN operator

```
71 •  SELECT *
72    FROM employees
73    WHERE salary BETWEEN 40000 AND 60000;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

emp_id	emp_name	department_id	salary	city
101	Arun	1	55000	Chennai
102	Priya	2	40000	Bangalore
104	Divya	3	45000	Coimbatore
106	Meena	2	55000	Chennai
107	Suresh	3	60000	Chennai
109	Kumar	4	48000	Chennai
110	Lakshmi	3	52000	Bangalore

Description: The query filters employees whose salary is BETWEEN 40000 AND 60000. BETWEEN includes the specified boundary values, so records within the salary range are returned. The result grid shows multiple employees whose salaries fall in this range. This demonstrates filtering numeric data using a defined range.

Screenshot 3 – UPDATE statement

```
61 •  UPDATE employees
62    SET salary = 55000
63    WHERE emp_id = 101;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

emp_id	emp_name	department_id	salary	city
101	Arun	1	55000	Chennai
102	Priya	2	40000	Bangalore
103	Ravi	1	65000	Chennai
104	Divya	3	45000	Coimbatore
105	Kiran	1	75000	Bangalore
106	Meena	2	55000	Chennai
107	Suresh	3	60000	Chennai
108	Anitha	4	70000	Madurai
109	Kumar	4	48000	Chennai
110	Lakshmi	3	52000	Bangalore

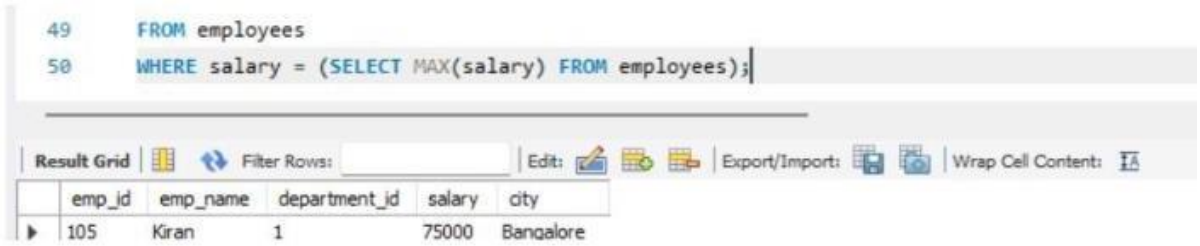
Description: The UPDATE statement changes the salary of the employee whose emp_id is 101. The statement sets salary = 55000 WHERE emp_id = 101. The result grid shows Arun's salary as 55000 after the update. This demonstrates how an existing record can be modified using UPDATE with a specific condition.

Screenshot 4 – SUM aggregate function



Description: The query SELECT SUM(salary) AS total_salary FROM employees calculates the combined salary of the employee records. The result grid displays total_salary as 560000. This demonstrates the SUM aggregate function and the use of an alias to give the calculated column a meaningful name.

Screenshot 5 – MAX function with subquery



Description: The query compares each employee's salary with the maximum salary returned by the subquery SELECT MAX(salary) FROM employees. The result identifies Kiran (emp_id 105) with a salary of 75000. This demonstrates the use of an aggregate function inside a subquery to retrieve the record matching the highest salary.

Screenshot 6 – ORDER BY

```
44 • SELECT *
45   FROM employees
46   ORDER BY salary ASC;
47
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

emp_id	emp_name	department_id	salary	city
102	Priya	2	40000	Bangalore
104	Divya	3	45000	Coimbatore
109	Kumar	4	48000	Chennai
101	Arun	1	50000	Chennai
110	Lakshmi	3	52000	Bangalore
106	Meena	2	55000	Chennai
107	Suresh	3	60000	Chennai
103	Ravi	1	65000	Chennai
108	Anitha	4	70000	Madurai
105	Kiran	1	75000	Bangalore

Description: The query SELECT * FROM employees ORDER BY salary ASC sorts the employee records by salary in ascending order. The result begins with the lower salaries and progresses toward the higher salaries, with Kiran's 75000 salary appearing at the end. This demonstrates how ORDER BY can organize query results using a numeric column.

Screenshot 7 – Salary comparison filter

```
19 • select * from employees where Salary >50000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID	Name	Department	Salary	Age	City	Year
103	Ravi	IT	65000	30	Chennai	2020
105	Kiran	IT	75000	32	Bangalore	2019
106	Meena	HR	55000	29	Chennai	2022
107	Suresh	Sales	60000	35	Chennai	2018
108	Anitha	Finance	70000	31	Madurai	2020
110	Lakshmi	Sales	52000	30	Bangalore	2021

Description: The query SELECT * FROM employees WHERE Salary > 50000 returns employees whose salary is greater than 50000. The result grid shows the matching employees along with their ID, Name, Department, Salary, Age, City, and Year. This demonstrates the use of the greater-than comparison operator for filtering records.

Screenshot 8 – Selecting specific columns

```
18 •  select Name,Department,Salary from employees;
19
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Name	Department	Salary
Arun	IT	50000
priya	HR	40000
Ravi	IT	65000
Divya	Sales	45000
Kiran	IT	75000
Meena	HR	55000
Suresh	Sales	60000
Anitha	Finance	70000
Kumar	Finance	48000
Lakshmi	Sales	52000

Description: The query SELECT Name, Department, Salary FROM employees retrieves only three selected columns instead of all fields in the table. The result grid displays each employee's name, department, and salary. This demonstrates column selection and helps produce focused query output.

Screenshot 9 – INSERT statement

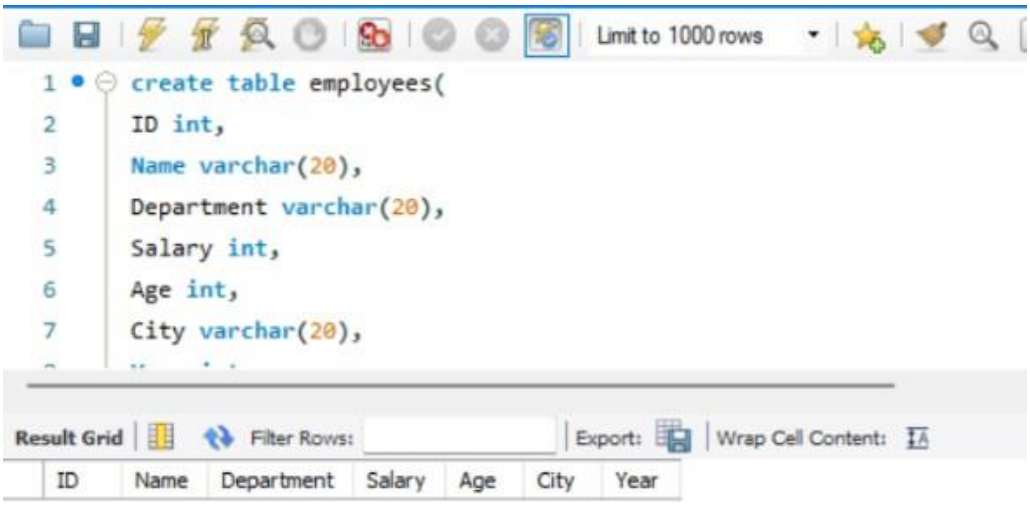
```
13  insert into employees(ID,Name,Department,Salary,Age,City,Year) values(101,'Arun','IT',50000,25,'Chennai',2022),(102,'priya','HR',400
14  (103,'Ravi','IT',65000,30,'Chennai',2020),(104,'Divya','Sales',45000,26,'Coimbatore',2023),(105,'Kiran','IT',75000,32,'Bangalore',20
15  (106,'Meena','HR',55000,29,'Chennai',2022),(107,'Suresh','Sales',60000,35,'Chennai',2018),(108,'Anitha','Finance',70000,31,'Madurai'
16  2020),(109,'Kumar','Finance',48000,27,'Chennai',2023),(110,'Lakshmi','Sales',52000,30,'Bangalore',2021);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID	Name	Department	Salary	Age	City	Year
101	Arun	IT	50000	25	Chennai	2022
102	priya	HR	40000	28	Bangalore	2021
103	Ravi	IT	65000	30	Chennai	2020
104	Divya	Sales	45000	26	Coimbatore	2023
105	Kiran	IT	75000	32	Bangalore	2019
106	Meena	HR	55000	29	Chennai	2022
107	Suresh	Sales	60000	35	Chennai	2018
108	Anitha	Finance	70000	31	Madurai	2020
109	Kumar	Finance	48000	27	Chennai	2023
110	Lakshmi	Sales	52000	30	Bangalore	2021

Description: The INSERT statement adds employee records to the employees table using the columns ID, Name, Department, Salary, Age, City, and Year. The result grid below the statement shows the inserted employee data. This demonstrates how multiple employee records can be stored in a relational table.

Screenshot 10 – CREATE TABLE statement



Description: The CREATE TABLE statement defines the employees table and its columns. The screenshot visibly shows fields including ID int, Name varchar(20), Department varchar(20), Salary int, Age int, and City varchar(20), with the table result grid displaying the column headings. This demonstrates the table-definition stage of the SQL practical.

5. Key SQL Concepts Demonstrated

SQL Concept	Purpose
CREATE	Defines the structure of the employees table and its columns.
INSERT	Adds employee records to the table.
SELECT	Retrieves data from the employees table.
WHERE	Restricts the result to records satisfying a condition.
LIKE	Performs pattern matching on text values.
BETWEEN	Filters values within a specified range.
ORDER BY	Sorts query results in ascending or descending order.
SUM()	Calculates the total of numeric values.
MAX()	Finds the highest numeric value.
Subquery	Uses the result of one query inside another query.
UPDATE	Modifies existing records in the table.

6. Conclusion

The practical work demonstrates the basic SQL operations required to create and manage an employee database. The screenshots provide evidence of table creation, data insertion, record retrieval, conditional filtering, sorting, aggregation, subqueries, and record updates. Together, these examples show how SQL can be used to store employee information and retrieve meaningful results from the database.